

Advance Python Programming

1)Introduction to the print() function in Python.

Ans:- The print() function is used to display output on the screen. It helps show text, numbers, or variable values to the user.

Example:

```
print("Hello, World!")
```

Output:

Hello, World!

2)Formatting outputs using f-strings and format().

Ans:- **f-strings:**

Used to insert variables directly inside a string. (Python 3.6+)

Example:

```
name = "Sahil"  
age = 20  
print(f"My name is {name} and I am {age} years old.")
```

Output:

My name is Sahil and I am 20 years old.

format():

Used to format strings by replacing {} placeholders.

Example:

```
name = "Sahil"  
age = 20  
print("My name is {} and I am {} years old.".format(name, age))
```

Output:

My name is Sahil and I am 20 years old.

3) Using the input() function to read user input from the keyboard.

Ans:- The input() function is used to take input from the user through the keyboard.

Example:

```
name = input("Enter your name: ")  
print("Hello,", name)
```

Output:

Enter your name: Sahil
Hello, Sahil

4) Converting user input into different data types (e.g., int, float, etc.).

Ans:- The input() function always returns data as a **string**. We use type conversion functions like int(), float(), etc., to convert it into other data types.

Example:

```
age = int(input("Enter your age: "))  
height = float(input("Enter your height: "))  
  
print(age)  
print(height)
```

5) Opening files in different modes ('r', 'w', 'a', 'r+', 'w+').

Ans:- Python uses the open() function to open files in different modes.

Mode Meaning	
'r'	Read file (default mode)
'w'	Write file (creates or overwrites file)
'a'	Append data to the end of file
'r+'	Read and write file
'w+'	Read and write (overwrites file)

Example:

```
file = open("data.txt", "r") # Read
```

```
file = open("data.txt", "w") # Write
```

```
file = open("data.txt", "a") # Append
```

```
file = open("data.txt", "r+") # Read and Write
```

```
file = open("data.txt", "w+") # Write and Read
```

6)Using the open() function to create and access files.

Ans:- open() function is used to create, open, read, and write files in Python.

Syntax:

```
file = open("filename.txt", "mode")
```

Example:

```
# Create and write to a file
```

```
file = open("data.txt", "w")
```

```
file.write("Hello Python")
```

```
file.close()
```

```
# Read the file
file = open("data.txt", "r")
print(file.read())
file.close()
```

7) Closing files using close().

Ans:- The close() method is used to close a file after completing file operations. It frees system resources and saves changes properly.

Example:

```
file = open("data.txt", "r")
print(file.read())
file.close()
```

8) Reading from a file using read(), readline(), readlines().

Ans:- **1. read()**

Reads the entire file content.

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

2. readline()

Reads one line at a time.

```
file = open("data.txt", "r")
line = file.readline()
print(line)
file.close()
```

3. readlines()

Reads all lines and returns them as a list.

```
file = open("data.txt", "r")
lines = file.readlines()
print(lines)
file.close()
```

9) Writing to a file using write() and writelines().

Ans:- **1. write()**

Writes a string to a file.

```
file = open("data.txt", "w")
file.write("Hello World")
file.close()
```

2. writelines()

Writes multiple lines from a list to a file.

```
file = open("data.txt", "w")
lines = ["Python\n", "Java\n", "C++\n"]
file.writelines(lines)
file.close()
```

10) Introduction to exceptions and how to handle them using try, except, and finally.

Ans:- An **exception** is an error that occurs during program execution. Exception handling prevents the program from crashing and allows it to continue running.

Syntax

```
try:
    # Code that may cause an error
except:
    # Code to handle the error
finally:
    # Code that always executes
```

Example

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print(result)

except ZeroDivisionError:
    print("Cannot divide by zero.")

except ValueError:
    print("Please enter a valid number.")

finally:
    print("Program finished.")
```

Explanation

- **try** → Contains code that may generate an exception.
- **except** → Handles the exception if an error occurs.
- **finally** → Always executes, whether an exception occurs or not.

Output Example

If input is 0:

Cannot divide by zero.

Program finished.

11) Understanding multiple exceptions and custom exceptions.

Ans:- **1. Multiple Exceptions**

A program can handle different types of exceptions using multiple except blocks.

try:

```
num = int(input("Enter a number: "))  
result = 10 / num
```

except ValueError:

```
print("Invalid input!")
```

except ZeroDivisionError:

```
print("Cannot divide by zero!")
```

Output:

- If input is text → Invalid input!
- If input is 0 → Cannot divide by zero!

2. Custom Exceptions

A custom exception is a user-defined exception created using a class.

```
class NegativeNumberError(Exception):
```

```
    pass
```

try:

```
num = int(input("Enter a positive number: "))
```

```
if num < 0:
```

```
    raise NegativeNumberError("Negative numbers are not  
allowed!")
```

```
except NegativeNumberError as e:  
    print(e)
```

Output:

Negative numbers are not allowed!

12) Understanding the concepts of classes, objects, attributes, and methods in Python.

Ans:- 1. Class

A **class** is a blueprint for creating objects.

```
class Student:  
    pass
```

2. Object

An **object** is an instance of a class.

```
class Student:  
    pass
```

```
s1 = Student()
```

Here, s1 is an object of the Student class.

3. Attributes

Attributes are variables that store data about an object.

```
class Student:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```



```
s1 = Student("Sahil", 20)
```

```
print(s1.name)
```

```
print(s1.age)
```

Output:

Sahil

20

4. Methods

Methods are functions defined inside a class that describe the object's behavior.

```
class Student:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def display(self):
```

```
        print("Student Name:", self.name)
```

```
s1 = Student("Sahil")
```

```
s1.display()
```

Output:

Student Name: Sahil

13) Difference between local and global variables.

Ans:-

Feature	Local Variable	Global Variable
Definition	Declared inside a function	Declared outside all functions
Scope	Accessible only within the function	Accessible throughout the program
Lifetime	Exists only while the function is running	Exists until the program ends
Declaration Location	Inside a function	Outside a function
Access	Cannot be accessed outside the function	Can be accessed inside and outside functions
Modification	Modified directly within the function	To modify inside a function, use the global keyword

14)Single, Multilevel, Multiple, Hierarchical, and Hybrid inheritance in Python.

Ans:- **1. Single Inheritance**

Single inheritance occurs when one child class inherits from one parent class.

```
class Parent:
    def show(self):
        print("Parent Class")
```

```
class Child(Parent):
    pass
```

```
c = Child()
c.show()
```

2. Multilevel Inheritance

Multilevel inheritance occurs when a class inherits from another derived class.

```
class GrandParent:
    def show(self):
        print("GrandParent Class")
```

```
class Parent(GrandParent):
    pass
```

```
class Child(Parent):
    pass
```

```
c = Child()
c.show()
```

3. Multiple Inheritance

Multiple inheritance occurs when one child class inherits from more than one parent class.

```
class Father:
    def father_method(self):
        print("Father Class")
```

```
class Mother:
    def mother_method(self):
        print("Mother Class")
```

```
class Child(Father, Mother):  
    pass
```

```
c = Child()  
c.father_method()  
c.mother_method()
```

4. Hierarchical Inheritance

Hierarchical inheritance occurs when multiple child classes inherit from a single parent class.

```
class Parent:  
    def show(self):  
        print("Parent Class")
```

```
class Child1(Parent):  
    pass
```

```
class Child2(Parent):  
    pass
```

```
c1 = Child1()  
c2 = Child2()
```

```
c1.show()  
c2.show()
```

5. Hybrid Inheritance

Hybrid inheritance is a combination of two or more types of inheritance.

```
class A:
    def show(self):
        print("Class A")
```

```
class B(A):
    pass
```

```
class C(A):
    pass
```

```
class D(B, C):
    pass
```

```
d = D()
d.show()
```

15) Using the `super()` function to access properties of the parent class.

Ans:- The `super()` function is used to access the methods and properties of the parent class from a child class.

Example

```
class Parent:
    def __init__(self, name):
        self.name = name
```

```
class Child(Parent):
    def __init__(self, name, age):
        super().__init__(name) # Call parent constructor
        self.age = age
```

```
def display(self):
    print("Name:", self.name)
```

```
print("Age:", self.age)
```

```
c = Child("Sahil", 20)
c.display()
```

Output

```
Name: Sahil
Age: 20
```

16) Method overloading: defining multiple methods with the same name but different parameters.

Ans:- **method Overloading** means defining multiple methods with the same name but different parameters.

Python does **not support true method overloading** like Java or C++. Instead, we can achieve similar behavior using default arguments or variable-length arguments (*args).

Example using Default Arguments

```
class Demo:
    def add(self, a, b=0, c=0):
        print(a + b + c)
```

```
d = Demo()
```

```
d.add(10)      # 10
d.add(10, 20)   # 30
d.add(10, 20, 30) # 60
```

Example using *args

```
class Demo:
    def add(self, *args):
        print(sum(args))
```

```
d = Demo()
```

```
d.add(10)
```

```
d.add(10, 20)
```

```
d.add(10, 20, 30)
```

17) Method overriding: redefining a parent class method in the child class.

Ans:- Method overriding occurs when a child class provides its own implementation of a method that already exists in the parent class.

Example:

```
class Parent:
```

```
    def show(self):
```

```
        print("Parent Class")
```

```
class Child(Parent):
```

```
    def show(self):
```

```
        print("Child Class")
```

```
c = Child()
```

```
c.show()
```

Output:

Child Class

18) Introduction to SQLite3 and PyMySQL for database connectivity.

Ans:- **SQLite3**

- **SQLite3** is a lightweight, file-based database included with Python.

- It does not require a separate server.
- Used for small applications and testing.

Example:

```
import sqlite3

conn = sqlite3.connect("student.db")
print("Database Connected")
conn.close()
```

PyMySQL

- **PyMySQL** is a Python library used to connect Python programs with a MySQL database.
- MySQL server must be installed and running.

Example:

```
import pymysql

conn = pymysql.connect(
    host="localhost",
    user="root",
    password="password",
    database="studentdb"
)

print("Database Connected")
conn.close()
```

19) Creating and executing SQL queries from Python using these connectors.

Ans:- Creating and Executing SQL Queries in Python

Using SQLite3

```
import sqlite3
```

```
conn = sqlite3.connect("student.db")
```

```
cursor = conn.cursor()
```

```
cursor.execute("CREATE TABLE Student(id INTEGER, name TEXT)")
```

```
cursor.execute("INSERT INTO Student VALUES(1, 'Sahil')")
```

```
conn.commit()
```

```
print("Data Inserted")
```

```
conn.close()
```

Using PyMySQL

```
import pymysql
```

```
conn = pymysql.connect(
```

```
    host="localhost",
```

```
    user="root",
```

```
    password="password",
```

```
    database="studentdb"
```

```
)
```

```
cursor = conn.cursor()
```

```
cursor.execute("INSERT INTO Student VALUES(1, 'Sahil')")
```

```
conn.commit()
```

```
print("Data Inserted")
```

```
conn.close()
```

20) Using `re.search()` and `re.match()` functions in Python's `re` module for pattern matching.

Ans:- The **re** module is used for pattern matching with regular expressions.

1. `re.search()`

- Searches the entire string for a pattern.
- Returns the first match found.

```
import re
```

```
text = "I am learning Python"
```

```
result = re.search("Python", text)
```

```
if result:
```

```
    print("Match Found")
```

Output:

Match Found

2. `re.match()`

- Checks for a match only at the beginning of the string.

```
import re
```

```
text = "Python is easy"
```

```
result = re.match("Python", text)
```

```
if result:  
    print("Match Found")
```

Output:

Match Found

21) Difference between search and match.

Ans:-

Feature	re.search()	re.match()
Meaning	Searches the entire string for a pattern	Checks for a pattern only at the beginning of the string
Search Area	Whole string	Start of the string only
Match Position	Can find a match anywhere	Must find a match at position 0
Return Value	Returns first match object if found	Returns match object only if pattern is at the start
Use Case	Find a pattern anywhere in text	Verify that text starts with a pattern